

VITyarthi

BUILD YOUR OWN PROJECT

PROJECT REPORT

BMI CALCULATOR

Submitted By

MAHIR RAHMAN

Registration No.: 28MIM10037

Technology: Python 3

Project Type: Python Programming Project

A console-based BMI calculation and classification application for adults and children.

1. INTRODUCTION

The BMI Calculator is a Python-based console application created to demonstrate fundamental programming concepts in a practical context. It accepts age, height and weight, and for children also accepts gender. It calculates BMI and displays a category according to the logic implemented in the supplied source code.

The program has separate adult and child paths. Users aged 20 or above are routed to the adult calculator, users between 3 and 19 are routed to the child calculator, and users aged 2 or below receive a message that BMI is not calculated for that age group.

For children, separate reference dictionaries are provided for boys and girls. The program retrieves four age-specific reference values and estimates a percentile using interpolation. The VITyarthi guidelines require a project to identify a meaningful problem, design a technical solution and document the implementation and evaluation. [filecite]turn1file0[L14-L20]

2. PROBLEM STATEMENT

A user may want a quick way to calculate BMI without manually performing the formula. A child also needs age- and gender-specific reference values for the interpretation used by this program. The project therefore provides a simple command-line solution that accepts user information, performs the required calculations, and displays the resulting classification.

Objectives

- Create a simple and understandable BMI calculator using Python.
- Separate adult and child BMI processing.
- Use dictionaries to organize child reference data.
- Estimate child BMI percentile using the stored reference points.
- Demonstrate functions, conditional logic, input/output and validation.

3. FUNCTIONAL REQUIREMENTS

The VITyarthi instructions require at least three major functional modules, a clear input/output structure and a logical workflow. [filecite\[turn1file0\[L24-L37\]](#)

Module	Description	Input / Output
Age Router	Determines which BMI path should be used.	Age → adult/child/message
Adult BMI	Calculates BMI and displays adult classification.	Height + weight → BMI/category
Child BMI	Calculates BMI and estimates percentile from reference data.	Gender + age + height + weight → percentile/category
Repeat Control	Allows another calculation after the first run.	Yes/No → restart/exit

4. NON-FUNCTIONAL REQUIREMENTS

The guidelines require at least four non-functional requirements. [filecite\[turn1file0\[L38-L50\]](#)

Requirement	Application in Project
Usability	Simple prompts and a direct console interface.
Reliability	Separated functions and predefined reference dictionaries.
Error Handling	Handles unsupported ages and invalid gender input; measurement validation is included in the supplied code.
Maintainability	Adult and child logic are kept in separate functions.
Resource Efficiency	Runs locally without external services or a database.

5. TECHNICAL EXPECTATIONS

- The implementation uses Python functions, conditional statements and dictionaries.
- The source contains separate functions for routing, adult BMI and child BMI.
- The child reference values are organized by age and gender.
- The project can be tested through console input cases.
- The VITyarthi guidelines emphasize modular implementation, documentation, validation/error handling and testing for coding projects. [filecite\[turn1file0\[L51-L63\]](#)

6. SYSTEM ARCHITECTURE

The system uses a simple procedural architecture. User input is collected through the console, the age router selects the appropriate module, the BMI calculation is performed, and the result is displayed.

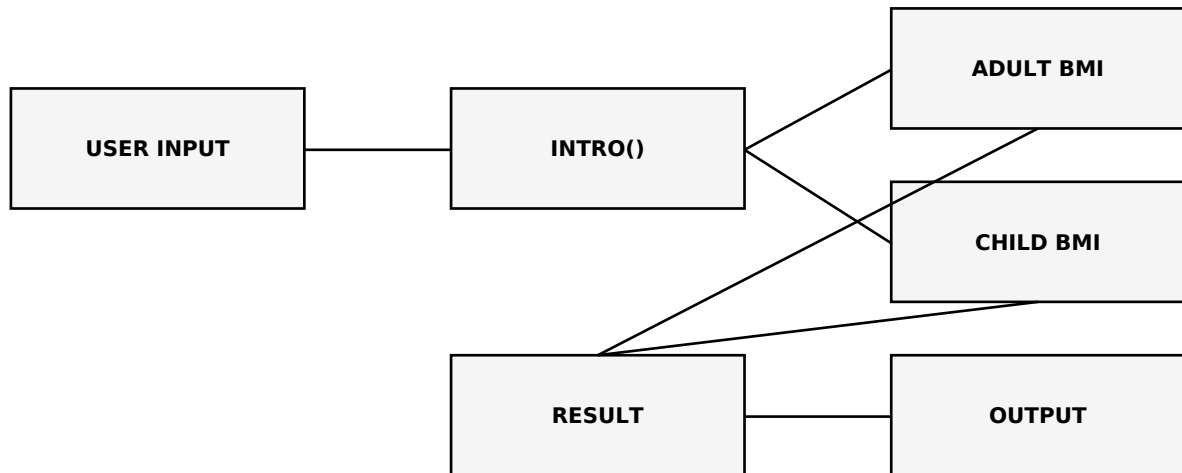


Figure 1. System architecture.

7. WORKFLOW

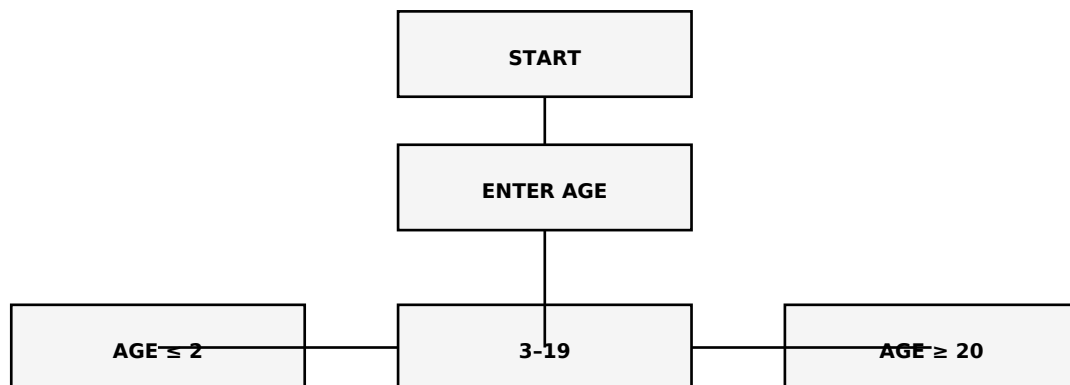


Figure 2. Simplified user workflow.

8. DESIGN DIAGRAMS

8.1 Component Diagram

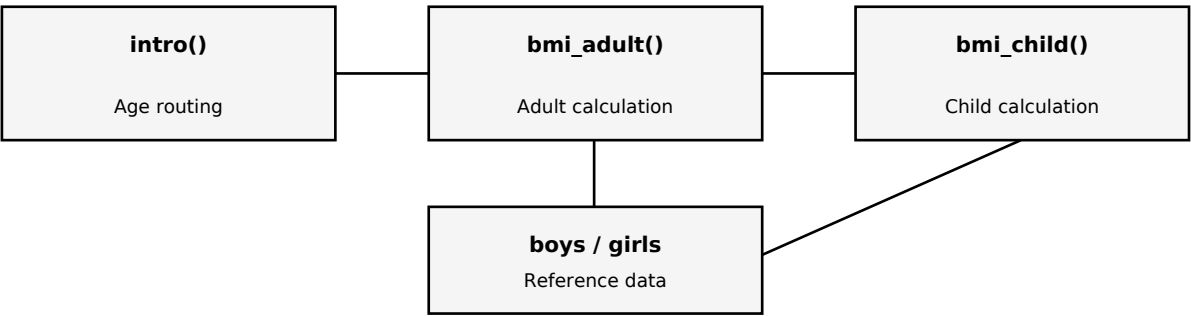


Figure 3. Program component diagram.

8.2 Sequence Diagram

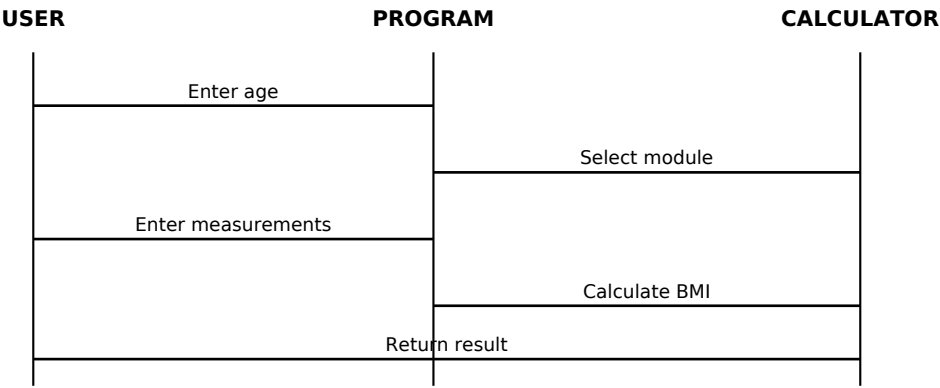


Figure 4. Simplified sequence diagram.

The report includes architecture and design diagrams because the VITyarthi report requirements explicitly list system architecture, use case/workflow, sequence and class/component diagrams.

filecite{turn1file0}{L66-L84}

9. IMPLEMENTATION DETAILS

The supplied source code begins with an `intro()` function that asks for age and chooses between the adult and child paths. The adult function receives height and weight and calculates BMI. The child function receives gender, height, weight and age, then selects the relevant reference dictionary.

`¶filecite¶turn0file0¶L3-L15¶`

BMI Formula

```
BMI = weight (kg) / height2 (m)
```

Adult Classification

The adult function compares the calculated BMI against the thresholds implemented in the source code and displays messages for underweight, fit, overweight and obese results.

`¶filecite¶turn0file0¶L19-L27¶`

Child Percentile Processing

The child function contains separate boys and girls dictionaries with age-specific values. It selects the dictionary according to the entered gender and retrieves p5, p50, p85 and p95 for the selected age.

`¶filecite¶turn0file0¶L37-L62¶`

The percentile is then estimated through conditional interpolation. The resulting category is set to Underweight, Healthy Weight, Overweight or Obese. `¶filecite¶turn0file0¶L63-L77¶`

10. DESIGN DECISIONS & RATIONALE

- Functions were used to separate major parts of the program and make the logic easier to follow.
- Dictionaries were used because each age needs a fixed set of reference values.
- Separate boys and girls dictionaries keep the child reference data organized.
- Interpolation provides an estimated percentile between stored reference points.
- A console interface keeps the project focused on Python programming fundamentals.
- No database is required because the current program does not persist user records.

11. SCREENSHOTS / RESULTS

The application is console based. Actual terminal screenshots from running the Python file should be inserted here before portal submission. The VITYarthi report requirements include screenshots/results. [filecite\turn1file0\L125-L129](#)

Scenario	Expected Result
Adult with BMI below 18.5	Displays BMI and Underweight message.
Adult with BMI 18.5–24.5	Displays BMI and Fit message.
Adult with higher BMI	Displays Overweight or Obese according to implemented conditions.
Child boy	Uses boys reference dictionary and estimates percentile.
Child girl	Uses girls reference dictionary and estimates percentile.
Invalid gender	Displays an instruction to enter boy or girl.
Age $\leq$ 2	Displays that BMI is not calculated for this age group.

12. TESTING APPROACH

Test	Input	Expected Behaviour
T1	Age = 20	Adult path selected.
T2	Age = 2	BMI not calculated.
T3	Age = 12, boy	Boys data selected.
T4	Age = 12, girl	Girls data selected.
T5	Invalid gender	Error message displayed.
T6	Positive measurements	BMI calculated.
T7	Repeat = Yes	Calculation starts again.
T8	Repeat = No	Program exits.

The VITYarthi guidelines require testing wherever applicable and validation/error handling for coding projects. [filecite\turn1file0\L56-L63](#)

13. CHALLENGES FACED

- Creating separate adult and child calculation paths.
- Organizing age-specific values in dictionaries.
- Handling gender selection correctly.
- Implementing percentile interpolation between reference values.
- Keeping the console interaction simple while supporting multiple cases.
- Considering invalid measurements and unsupported inputs.

14. LEARNINGS & KEY TAKEAWAYS

- Improved understanding of Python functions and program structure.
- Practiced conditional statements and decision-making logic.
- Learned how dictionaries can represent structured data.
- Learned how interpolation can estimate values between known points.
- Practiced input/output and validation.
- Understood the importance of testing and documentation.

15. FUTURE ENHANCEMENTS

- Add a graphical user interface using Tkinter.
- Improve validation for non-numeric, zero and negative measurements.
- Support centimetres and feet/inches for height.
- Add calculation history.
- Add charts for BMI and percentile.
- Use a more comprehensive validated child reference dataset.
- Add automated unit tests.
- Split the program into multiple Python modules.
- Add README.md, statement.md, source files and screenshots to the GitHub repository as required by the guidelines. [filecite turn1file0 L87-L106](#)

16. REFERENCES

1. VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines. The document was used as the structure and requirement basis for this report.

□filecite□turn1file0□L109-L134□

2. Supplied BMI Calculator Python source file. The implementation was used as the basis for the project description and technical sections. □filecite□turn0file0□L3-L15□

3. Boys and girls age-specific reference dictionaries and percentile logic contained in the supplied source code. □filecite□turn0file0□L37-L77□

Note: This report documents the supplied educational program. The medical validity of the reference dataset is not independently verified here.

17. CONCLUSION

The BMI Calculator demonstrates how Python can be used to solve a practical problem through structured programming. It contains separate adult and child processing, age-based routing, dictionary-based reference data, BMI calculations and child percentile estimation.

The project meets the core documentation structure requested for the Build Your Own Project report and provides clear opportunities for future improvement through stronger validation, a graphical interface, automated testing, improved data and a more modular project structure.

Student: MAHIR RAHMAN

Registration No.: 28MIM10037

Project: BMI Calculator

— END OF PROJECT REPORT —